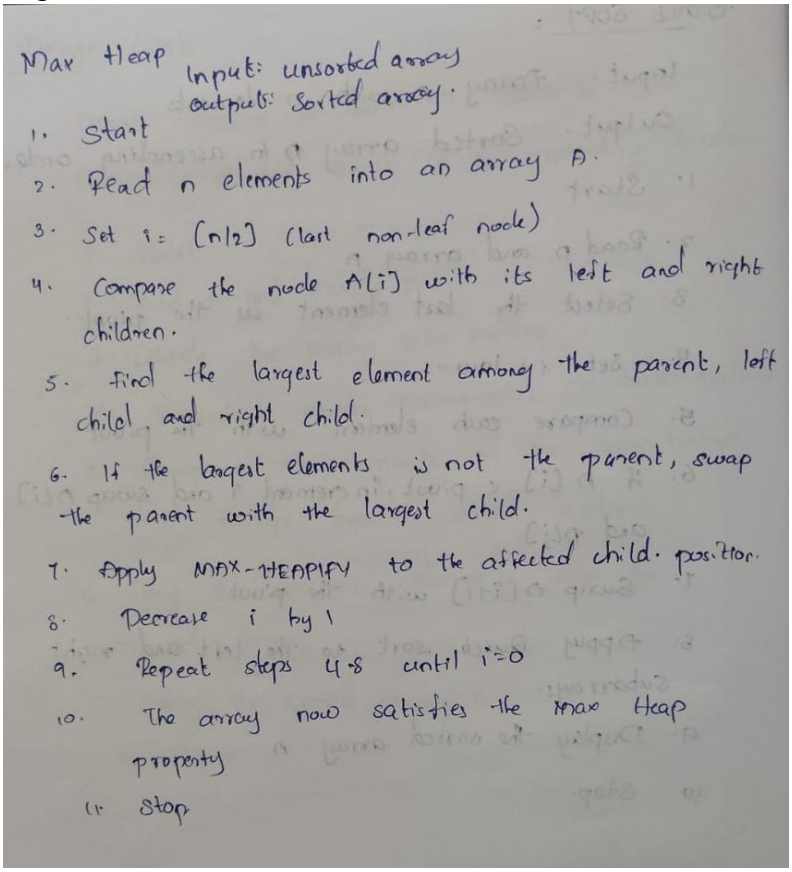
Practical – 3

Aim : To Implementation of Max-Heap sort algorithm

Algorithm :



Program / Code :

```
#include <iostream>
#include <vector>
using namespace std;

void heapify(vector<int> &arr, int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
```

```
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapsort(vector<int> &arr)
{
    int n = static_cast<int>(arr.size());

    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main()
{
    vector<int> arr = {64, 34, 25, 12, 22, 11, 90};

    heapsort(arr);

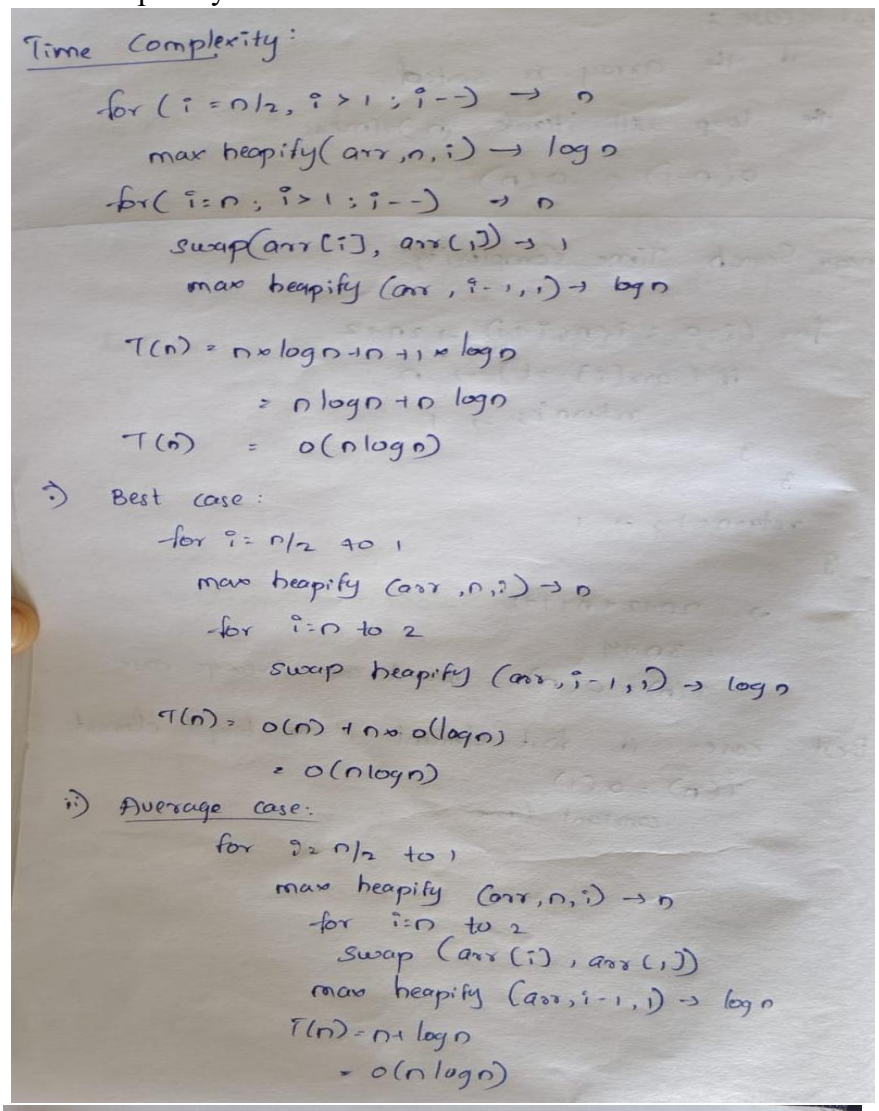
    cout << "Sorted array using heap sort:\n";
    for (int value : arr)
        cout << value << ' ';

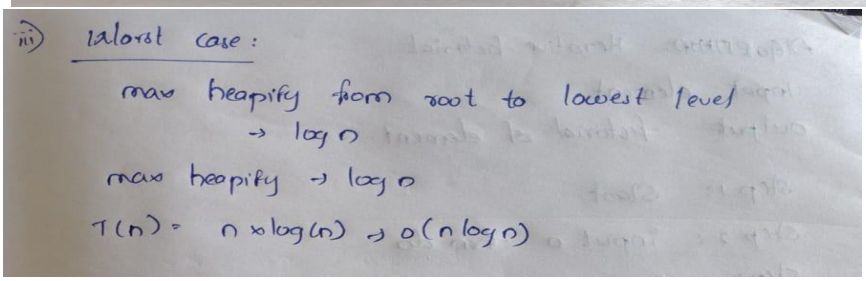
    cout << '\n';
    return 0;
}
```

OUTPUT:

```
PS C:\Users\pusap\Downloads\c++> C:\Users\pusap\Downloads\c++\heapsort.exe
Sorted array using heap sort:
11 12 22 25 34 64 90
```

Time Complexity :





Conclusion :

The Max-Heap Sort algorithm was successfully implemented and executed to sort the elements in ascending order. The program verified a consistent time complexity of \(O(n \log n)\) across the best, average, and worst cases using an in-place binary heap structure